Mini Project 9 — Membangun Dataset dan Membekukan Split Eksperimen

Mini Project 9 adalah tahap pengumpulan dataset utama running project. Mahasiswa merekam enam acquisition block; setiap block berisi empat perintah. Skrip kemudian menerapkan filter MP6, preprocessing MP7, dan feature schema MP8 secara otomatis, menyimpan metadata dan log-Mel, lalu membuat grouped split yang tidak boleh diubah pada MP10–MP13.

Tabel 9.12. Input dan output Mini Project 9.

Jenis	Path	Peran
Status input	project_status.json = ready_for_mp09	validasi serial
Config input	project/preprocessing/feature config	parameter yang sudah dikunci
Filter input	mp06_filter_final.npz	filter final
Raw output	data/raw/mp09_dataset/	24 WAV utama
Metadata	data/metadata/mp09_metadata.csv	label dan group
Dataset	features/dataset/mp09_features_39d.npz	X 24×39 + label/group
Log-Mel	features/logmel/mp09/*.npz	jalur deep learning
Split	splits/mp09_split.json	train/validation/test IDs
Status output	project_status.json = ready_for_mp10	handoff serial

Prosedur Step by Step

Langkah 1 — Validasi status MP8

Buka terminal pada root `audio_ai_embedded_project`. Pastikan status proyek bernilai ready_for_mp09 dan feature_config.json tersedia.

```
python -c "import json, pathlib; s=json.load(open('project_status.json')); print(s['status']); assert s['status']=='ready_for_mp09'; assert pathlib.Path('config/feature_config.json').exists()"
```

Langkah 2 — Pastikan paket tersedia

Paket NumPy, SciPy, Matplotlib, dan sounddevice digunakan. Jika sudah terpasang dari MP1, langkah ini hanya verifikasi.

```
python -m pip install numpy scipy matplotlib sounddevice
```

Langkah 3 — Simpan skrip MP9

Buat file `mini_project_09.py` pada root proyek dan salin kode Python di bawah. Biarkan DEMO_MODE=False.

Langkah 4 — Rekam enam acquisition block

Jalankan skrip. Pada setiap block, ucapkan nyala, mati, naik, dan turun satu kali. Setelah satu block selesai, berhenti 5–10 detik dan ubah posisi mikrofon sedikit. Jangan mengubah sampling rate.

```
python mini_project_09.py
```

Langkah 5 — Tunggu ekstraksi otomatis

Skrip menerapkan filter, preprocessing, 39-D feature schema, menyimpan log-Mel, lalu membuat grouped split.

Langkah 6 — Periksa metadata dan split

Pastikan tiap kelas memiliki enam sampel; train berisi 16, validation empat, test empat; group tidak overlap.

```
python -c "import json; m=json.load(open('results/metrics/mp09_dataset_metrics.json')); print(m); assert m['n_samples']==24 and m['feature_dim']==39
and not m['group_overlap_train_test']"
```

Langkah 7 — Validasi handoff MP10

Pastikan project_status.json berubah menjadi ready_for_mp10 serta dataset dan split tercatat.

> **Cara merekam 24 sampel dalam waktu singkat**
>
> Satu acquisition block hanya membutuhkan empat rekaman 1,2 detik. Fokus pada pengucapan natural dan konsisten. Perubahan kecil posisi antarblock sengaja dibuat untuk menghasilkan variasi channel sederhana sekaligus memberi unit grouping. Jangan sengaja mengubah label atau menggunakan efek audio.

Kode Python Mini Project 9

```
from pathlib import Path
import csv, json, time
import numpy as np
import matplotlib.pyplot as plt
from scipy.io import wavfile
from scipy.signal import lfilter
from scipy.fft import dct

ROOT = Path('.')
STATUS = ROOT / 'project_status.json'
DEMO_MODE = False      # True hanya untuk pengujian tanpa mikrofon
N_BLOCKS = 6
DURATION_S = 1.2
SEED = 2026
rng = np.random.default_rng(SEED)

status = json.loads((ROOT/'project_status.json').read_text(encoding='utf-8'))
assert status['status'] == 'ready_for_mp09', status['status']

project_cfg = json.loads((ROOT/'config/project_config.json').read_text(encoding='utf-8'))
pre_cfg = json.loads((ROOT/'config/preprocessing_config.json').read_text(encoding='utf-8'))
feat_cfg = json.loads((ROOT/'config/feature_config.json').read_text(encoding='utf-8'))
filter_npz = np.load(ROOT/'features/intermediate/mp06_filter_final.npz', allow_pickle=True)

FS = int(project_cfg.get('sampling_rate_hz', 16000))
LABELS = project_cfg.get('labels', ['nyala','mati','naik','turun'])
B = np.asarray(filter_npz['b'], dtype=float)
A = np.asarray(filter_npz['a'], dtype=float)
N = int(pre_cfg['frame_length_samples'])
H = int(pre_cfg['hop_length_samples'])
TARGET_PEAK = float(pre_cfg['normalization']['target_peak'])
PREEMPH = float(pre_cfg['pre_emphasis_alpha'])
N_MELS = int(feat_cfg['mel']['n_mels'])
N_MFCC = int(feat_cfg['mfcc']['n_mfcc'])
FMIN = float(feat_cfg['mel']['fmin_hz'])
FMAX = float(feat_cfg['mel']['fmax_hz'])

raw_dir = ROOT/'data/raw/mp09_dataset'
meta_dir = ROOT/'data/metadata'
dataset_dir = ROOT/'features/dataset'
logmel_dir = ROOT/'features/logmel/mp09'
split_dir = ROOT/'splits'
metric_dir = ROOT/'results/metrics'
fig_dir = ROOT/'results/figures'
for d in [raw_dir,meta_dir,dataset_dir,logmel_dir,split_dir,metric_dir,fig_dir]: d.mkdir(parents=True, exist_ok=True)

def save_wav(path, y):
    y = np.clip(y, -1, 1)
    wavfile.write(path, FS, np.round(y*32767).astype(np.int16))

def acquire(label, block):
    path = raw_dir/f'{label}_b{block:02d}.wav'
    if path.exists(): return path
```

```
    if DEMO_MODE:
        # synthetic fallback hanya untuk test script
        f0 = {'nyala':170,'mati':205,'naik':240,'turun':145}[label]
        t = np.arange(int(DURATION_S*FS))/FS
        env = np.sin(np.pi*np.clip(t/DURATION_S,0,1))**1.4
        y = env*(0.45*np.sin(2*np.pi*f0*t)+0.16*np.sin(2*np.pi*2.3*f0*t))
        y += 0.012*rng.normal(size=len(t)) + 0.006*(block-1)*np.sin(2*np.pi*50*t)
    else:
        import sounddevice as sd
        input(f'Block {block} | ucapkan "{label}" lalu tekan ENTER... ')
        print('Merekam...')
        y = sd.rec(int(DURATION_S*FS), samplerate=FS, channels=1, dtype='float32')
        sd.wait(); y = y[:,0]
    save_wav(path, y)
    return path

def read_float(path):
    fs,y = wavfile.read(path)
    assert fs == FS, (path,fs)
    if y.ndim>1: y=y.mean(axis=1)
    if np.issubdtype(y.dtype,np.integer): y=y.astype(np.float64)/np.iinfo(y.dtype).max
    else: y=y.astype(np.float64)
    return y

def hz_to_mel(f): return 2595*np.log10(1+np.asarray(f)/700.0)
def mel_to_hz(m): return 700*(10**(np.asarray(m)/2595.0)-1)
def melbank(n_fft):
    m=np.linspace(hz_to_mel(FMIN),hz_to_mel(FMAX),N_MELS+2)
    hz=mel_to_hz(m); bins=np.floor((n_fft+1)*hz/FS).astype(int)
    fb=np.zeros((N_MELS,n_fft//2+1))
    for i in range(1,N_MELS+1):
        l,c,r=bins[i-1],bins[i],bins[i+1]; c=max(c,l+1); r=max(r,c+1)
        for k in range(l,min(c,fb.shape[1])): fb[i-1,k]=(k-l)/(c-l)
        for k in range(c,min(r,fb.shape[1])): fb[i-1,k]=(r-k)/(r-c)
    return fb
FB = melbank(N)
FREQ = np.fft.rfftfreq(N,1/FS)

def preprocess(y):
    # filter final MP6
    y=lfilter(B,A,y)
    y=y-np.mean(y)
    pk=np.max(np.abs(y))+1e-12; y=y*(TARGET_PEAK/pk)
    if len(y)<N: y=np.pad(y,(0,N-len(y)))
    starts=np.arange(0,len(y)-N+1,H)
    raw=np.array([y[s:s+N] for s in starts])
    rms=np.sqrt(np.mean(raw**2,axis=1)+1e-12)
    thr=float(pre_cfg['vad']['relative_factor'])*np.max(rms)
    active=np.where(rms>=thr)[0]
    if len(active):
        margin=int(pre_cfg['vad'].get('margin_frames',2))
        i0=max(0,int(active[0])-margin); i1=min(len(raw)-1,int(active[-1])+margin)
        y=y[starts[i0]:min(len(y),starts[i1]+N)]
    # denoise tidak dipaksakan bila MP7 menandai noise estimate tidak reliable
    if len(y)<N: y=np.pad(y,(0,N-len(y)))
    starts=np.arange(0,len(y)-N+1,H)
    fr=np.array([y[s:s+N] for s in starts])
    fr=np.concatenate([fr[:,:1], fr[:,1:]-PREEMPH*fr[:,:-1]],axis=1)
    return fr*np.hamming(N)[None,:]

def pitch_one(frame):
    x=frame-np.mean(frame); ac=np.correlate(x,x,mode='full')[len(x)-1:]
    if ac[0]<=1e-12: return 0.0,0.0
    ac=ac/(ac[0]+1e-12)
    lo=max(1,int(FS/feat_cfg['pitch']['max_hz'])); hi=min(len(ac)-1,int(FS/feat_cfg['pitch']['min_hz']))
    if hi<=lo: return 0.0,0.0
    lag=int(np.argmax(ac[lo:hi+1])+lo); conf=float(ac[lag]); f0=FS/lag
    if conf<float(feat_cfg['pitch']['confidence_threshold']): return 0.0,conf
    return float(f0),conf

def extract(path):
    y=read_float(path); frames=preprocess(y)
    rms=np.sqrt(np.mean(frames**2,axis=1)+1e-12)
    zcr=np.mean(np.abs(np.diff(np.signbit(frames),axis=1)),axis=1)
    spec=np.abs(np.fft.rfft(frames,axis=1))+1e-12; power=spec**2
    den=np.sum(power,axis=1)+1e-12
    centroid=np.sum(power*FREQ[None,:],axis=1)/den
    bw=np.sqrt(np.sum(power*(FREQ[None,:]-centroid[:,None])**2,axis=1)/den)
    cum=np.cumsum(power,axis=1); total=cum[:,-1]
    roll=np.array([FREQ[min(np.searchsorted(cum[i],.85*total[i]),len(FREQ)-1)] for i in range(len(frames))])
    pitch=np.array([pitch_one(f)[0] for f in frames])
    voiced=pitch>0
    mel=np.maximum(power@FB.T,1e-12); logmel=np.log(mel)
    mfcc=dct(logmel,type=2,axis=1,norm='ortho')[:,:N_MFCC]
    vals=[]; names=[]
    for j in range(N_MFCC):
        vals += [float(np.mean(mfcc[:,j])),float(np.std(mfcc[:,j]))]
        names += [f'mfcc{j+1:02d}_mean',f'mfcc{j+1:02d}_std']
    for name,a in [('rms',rms),('zcr',zcr),('centroid_hz',centroid),('bandwidth_hz',bw),('rolloff85_hz',roll)]:
        vals += [float(np.mean(a)),float(np.std(a))]; names += [name+'_mean',name+'_std']
    pv=pitch[voiced]
    vals += [float(np.mean(pv)) if len(pv) else 0.0,float(np.std(pv)) if len(pv) else 0.0,float(np.mean(voiced))]
    names += ['f0_hz_mean','f0_hz_std','voiced_ratio']
    assert len(vals)==39
    return np.asarray(vals,np.float32), np.asarray(names,dtype=object), logmel.astype(np.float32), len(y)/FS

# 1) Akuisisi 6 block × 4 kelas
records=[]
```

```
for block in range(1,N_BLOCKS+1):
    print(f'--- Acquisition block {block}/{N_BLOCKS} ---')
    for label in LABELS:
        p=acquire(label,block)
        records.append({'sample_id':p.stem,'label':label,'group_id':f'block_{block:02d}',
                        'subject_id':'personal_user','device_id':'local_mic','distance_tag':'normal',
                        'sample_rate_hz':FS,'path':p.as_posix()})
    if not DEMO_MODE and block<N_BLOCKS:
        print('Istirahat 5–10 detik / ubah posisi mikrofon sedikit sebelum block berikutnya.')
        time.sleep(2)

# 2) Ekstraksi feature schema MP8 untuk seluruh sampel
X=[]; y=[]; groups=[]; ids=[]; paths=[]; durations=[]; feature_names=None
for r in records:
    vec,names,logmel,dur=extract(Path(r['path']))
    if feature_names is None: feature_names=names
    else: assert np.array_equal(feature_names,names)
    X.append(vec); y.append(r['label']); groups.append(r['group_id']); ids.append(r['sample_id']); paths.append(r['path']); durations.append(dur)
    np.savez_compressed(logmel_dir/f"{r['sample_id']}.npz",log_mel=logmel, sample_id=r['sample_id'], label=r['label'])
    r['duration_s']=round(dur,4)

X=np.vstack(X); y=np.asarray(y,dtype=object); groups=np.asarray(groups,dtype=object); ids=np.asarray(ids,dtype=object); paths=np.asarray(paths,dtype=object)
assert X.shape==(N_BLOCKS*len(LABELS),39)
assert np.isfinite(X).all()

# 3) Grouped split: block 1–4 train, 5 validation, 6 test
train_mask=np.isin(groups,[f'block_{i:02d}' for i in range(1,5)])
val_mask=groups=='block_05'; test_mask=groups=='block_06'
assert not np.any(train_mask & val_mask) and not np.any(train_mask & test_mask)
assert set(groups[train_mask]).isdisjoint(set(groups[val_mask]))
assert set(groups[train_mask]).isdisjoint(set(groups[test_mask]))

def ids_of(mask): return ids[mask].tolist()
split={'strategy':'grouped_by_acquisition_block','seed':SEED,'train_ids':ids_of(train_mask),'validation_ids':ids_of(val_mask),'test_ids':ids_of(test_mask),
       'train_groups':sorted(set(groups[train_mask])),'validation_groups':sorted(set(groups[val_mask])),'test_groups':sorted(set(groups[test_mask]))}
(split_dir/'mp09_split.json').write_text(json.dumps(split,indent=2),encoding='utf-8')

# 4) Metadata dan dataset matrix
with (meta_dir/'mp09_metadata.csv').open('w',newline='',encoding='utf-8') as f:
    cols=['sample_id','label','group_id','subject_id','device_id','distance_tag','sample_rate_hz','duration_s','path','split']
    w=csv.DictWriter(f,fieldnames=cols); w.writeheader()
    for r in records:
        sid=r['sample_id']; sp='train' if sid in split['train_ids'] else ('validation' if sid in split['validation_ids'] else 'test')
        row=dict(r); row['split']=sp; w.writerow({k:row[k] for k in cols})
np.savez_compressed(dataset_dir/'mp09_features_39d.npz',X=X,y=y,groups=groups,sample_ids=ids,paths=paths,feature_names=feature_names,
                    train_mask=train_mask,val_mask=val_mask,test_mask=test_mask)

# 5) Config, metrik, visualisasi
cfg={'sampling_rate_hz':FS,'labels':LABELS,'n_blocks':N_BLOCKS,'samples_per_class':N_BLOCKS,'total_samples':len(ids),
     'record_duration_s':DURATION_S,'group_key':'group_id','split_strategy':'blocks_01_04_train__05_validation__06_test','seed':SEED,
     'feature_schema_source':'config/feature_config.json','feature_dim':39,'augmentation':'NONE_in_MP09__training_only_later'}
(ROOT/'config/dataset_config.json').write_text(json.dumps(cfg,indent=2),encoding='utf-8')

def class_counts(mask): return {lab:int(np.sum(y[mask]==lab)) for lab in LABELS}
metrics={'n_samples':int(len(ids)),'feature_dim':int(X.shape[1]),'finite_features':bool(np.isfinite(X).all()),
         'class_counts_all':class_counts(np.ones(len(y),dtype=bool)),'class_counts_train':class_counts(train_mask),
         'class_counts_validation':class_counts(val_mask),'class_counts_test':class_counts(test_mask),
         'n_train':int(train_mask.sum()),'n_validation':int(val_mask.sum()),'n_test':int(test_mask.sum()),
         'group_overlap_train_val':sorted(set(groups[train_mask])&set(groups[val_mask])),
         'group_overlap_train_test':sorted(set(groups[train_mask])&set(groups[test_mask]))}
(metric_dir/'mp09_dataset_metrics.json').write_text(json.dumps(metrics,indent=2),encoding='utf-8')

fig,ax=plt.subplots(figsize=(7,4)); ax.bar(LABELS,[metrics['class_counts_all'][l] for l in LABELS]); ax.set_ylabel('samples'); ax.set_title('MP9 class distribution');
ax.grid(axis='y',alpha=.2); fig.tight_layout(); fig.savefig(fig_dir/'mp09_class_distribution.png',dpi=160); plt.close(fig)
fig,ax=plt.subplots(figsize=(7,4)); ax.bar(['train','validation','test'],[train_mask.sum(),val_mask.sum(),test_mask.sum()]); ax.set_ylabel('samples'); ax.set_title('MP9 grouped
split'); ax.grid(axis='y',alpha=.2); fig.tight_layout(); fig.savefig(fig_dir/'mp09_split_distribution.png',dpi=160); plt.close(fig)

# 6) Handoff ke MP10
status.update({'status':'ready_for_mp10','mp10_dataset':'features/dataset/mp09_features_39d.npz','mp10_split':'splits/mp09_split.json',
               'dataset_config':'config/dataset_config.json','dataset_metadata':'data/metadata/mp09_metadata.csv','feature_config':'config/feature_config.json'})
status.setdefault('completed_mini_projects',[])
if 9 not in status['completed_mini_projects']: status['completed_mini_projects'].append(9)
STATUS.write_text(json.dumps(status,indent=2),encoding='utf-8')

print('MP9 selesai:', X.shape, 'train/val/test =', train_mask.sum(), val_mask.sum(), test_mask.sum())
print('status = ready_for_mp10')
```

Kriteria Lulus Mini Project 9

Tabel 9.14. Checklist kelulusan Mini Project 9.

Pemeriksaan	Kriteria lulus
Jumlah sampel	24 WAV; enam per kelas
Feature matrix	shape = 24 × 39 dan seluruh nilai finite

Pemeriksaan	Kriteria lulus
Grouped split	16 train, 4 validation, 4 test
Group overlap	kosong antar-split
Metadata	24 baris dengan label/group/path
Log-Mel	satu NPZ per sampel
Status	ready_for_mp10

Artefak yang Wajib Dibawa ke Mini Project 10

- `features/dataset/mp09_features_39d.npz` — matrix fitur, label, group, sample IDs, dan mask split.
- `splits/mp09_split.json` — split IDs yang tidak boleh diubah pada MP10–MP13.
- `config/dataset_config.json` — protokol dataset dan seed.
- `data/metadata/mp09_metadata.csv` — audit sumber dan grouping.
- `config/feature_config.json` — feature schema 39-D yang tetap.
- `project_status.json` dengan status `ready_for_mp10`.

Handoff ke Mini Project 10

Mini Project 10 akan melakukan seleksi fitur dan/atau reduksi dimensi. Semua proses yang belajar dari distribusi fitur, termasuk scaler, correlation threshold, mutual information, PCA, atau LDA, hanya boleh di-fit pada TRAIN menggunakan mask yang telah dibekukan oleh MP9.

Referensi Bab

1. Bishop, C. M. Pattern Recognition and Machine Learning. Springer.

2. Hastie, T., Tibshirani, R., dan Friedman, J. The Elements of Statistical Learning. Springer.

3. Géron, A. Hands-On Machine Learning with Scikit-Learn, Keras, and TensorFlow. O'Reilly.

4. Rabiner, L. R., dan Schafer, R. W. Theory and Applications of Digital Speech Processing. Pearson.

5. Sculley, D., et al. Hidden Technical Debt in Machine Learning Systems. Advances in Neural Information Processing Systems.

6. Scikit-learn documentation: model selection, cross-validation, preprocessing, and metrics.